

# **Software Requirements Specification**

## **for**

# **Motion Detection Surveillance System using Raspberry PI camera module**

**Version 3.0.0**

**Prepared by Mikołaj Żyszkiewicz**

**Lodz University of Technology**

**20.05.2026**

# Table of Contents

|                                                       |    |
|-------------------------------------------------------|----|
| 1. Introduction.....                                  | 4  |
| 1.1. Purpose .....                                    | 4  |
| 1.2. Document Conventions .....                       | 4  |
| 1.3. Intended Audience and Reading Suggestions .....  | 4  |
| 1.4. Product Scope ----.....                          | 4  |
| 1.5. References .....                                 | 4  |
| 2. Overall Description.....                           | 5  |
| 2.1. Product Perspective .....                        | 5  |
| 2.2. Product Features .....                           | 5  |
| 2.3. User Classes and Characteristics .....           | 7  |
| 2.4. Operating Environment.....                       | 7  |
| 2.5. Design and Implementation Constraints.....       | 7  |
| 2.6. User Documentation.....                          | 8  |
| 2.7. Assumptions and Dependencies.....                | 8  |
| 3. System Features.....                               | 8  |
| 3.1. Real-Time Motion Detection Engine.....           | 8  |
| 3.2. Automated Video Capture and Local Archiving..... | 8  |
| 3.3. Alert and Telemetry Dispatch.....                | 8  |
| 3.4. Rolling Media Lifecycle Management.....          | 8  |
| 3.5. Fault-Tolerant Power Recovery.....               | 9  |
| 3.6. Secure Configuration and Access Control.....     | 9  |
| 4. External Interface Requirements.....               | 9  |
| 4.1. User Interfaces Overview.....                    | 9  |
| 4.2. Hardware Interfaces.....                         | 9  |
| 4.3. Software Interfaces.....                         | 9  |
| 5. System Features/Modules.....                       | 10 |
| 5.1. «System Feature Name».....                       | 10 |

|                                         |    |
|-----------------------------------------|----|
| 5.1.1 Description and Priority.....     | 10 |
| 5.1.2 Stimulus/Response Sequences.....  | 10 |
| 5.1.3 Functional Requirements ----..... | 10 |
| 6. Nonfunctional Requirements ----..... | 10 |
| 6.1. Performance .....                  | 10 |
| 6.2. Security .....                     | 11 |
| 6.3. Reliability.....                   | 11 |

#### Revision History

| Name                | Date       | Reason for Changes                                                                     | Version |
|---------------------|------------|----------------------------------------------------------------------------------------|---------|
| Mikołaj Żyszkiewicz | 20.05.2026 | Initial version                                                                        | 1.0.0   |
| Mikołaj Żyszkiewicz | 02.06.2026 | Updated the product features and added a UML graph                                     | 2.0.0   |
| Mikołaj Żyszkiewicz | 08.06.2026 | Updated the System Features, Extrenal Interface Requirements and create a Class graph. | 3.0.0   |

## **1. Introduction**

### **1.1. Purpose**

«Identify the product whose software requirements and design are specified in this document, including the revision or release number. Describe the scope of the product that is covered by this specification.»

### **1.2. Document Conventions**

«Describe any standards or typographical conventions that were followed when writing this specification, such as fonts or highlighting that have special significance. For example, state whether priorities for higher-level requirements are assumed to be inherited by detailed requirements, or whether every requirement statement is to have its own priority.»

### **1.3. Intended Audience and Reading Suggestions**

«Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. Describe what the rest of this document contains and how it is organized. Suggest a sequence for reading the document, beginning with the overview sections and proceeding through the sections that are most pertinent to each reader type.»

### **1.4. Product Scope ----**

The software described herein will be used as the firmware for the Raspberry Pi based Motion Detection Surveillance System. The software is designed for autonomous video monitoring of an area in real-time, detecting threats in either interior or restricted areas with no need for ongoing human supervision or reliance upon remote cloud computing infrastructure. The firmware communicates with the camera module installed on the Raspberry Pi to perform environmental telemetry processing and event logging (capture) of digitally recorded evidence only when there are significant visual anomalies. The software performs all computer vision and data management tasks at the network edge to minimize notification time (latency), optimize the use of local storage space, and reduce energy consumption. Therefore, the software offers a secure and cost-effective alternative to commercially available security networks, consistent

with the design principles of embedded systems paradigm for stand-alone, edge-computed IoT deployments.

## 1.5. References

«List any other documents or Web addresses to which this document refers or was based on. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location. Include the assignment document in the references.»

## 2. Overall Description

### 2.1. Product Perspective

The Motion Detection Surveillance System specified in this document is a new, self-contained firmware application engineered to operate entirely at the network edge. It does not serve as a direct follow-on member of an existing product family, nor is it a legacy modification of a pre-existing software suite. Instead, it was conceived as a standalone solution to address distinct architectural vulnerabilities common in contemporary commercial security infrastructures.

Historically, localized surveillance has depended heavily on continuous human oversight or high-bandwidth cloud video recording (CVR) pipelines. These dependencies introduce single points of failure, including high operational subscription costs, severe latency bottlenecks, and significant data privacy risks. This product originates from the engineering need to decouple real-time visual threat detection from external networks. By shifting the computational workload to a low-cost, edge-computed hardware platform—specifically the Raspberry Pi architecture—the system serves as a decentralized replacement for traditional, cloud-dependent IP camera systems.

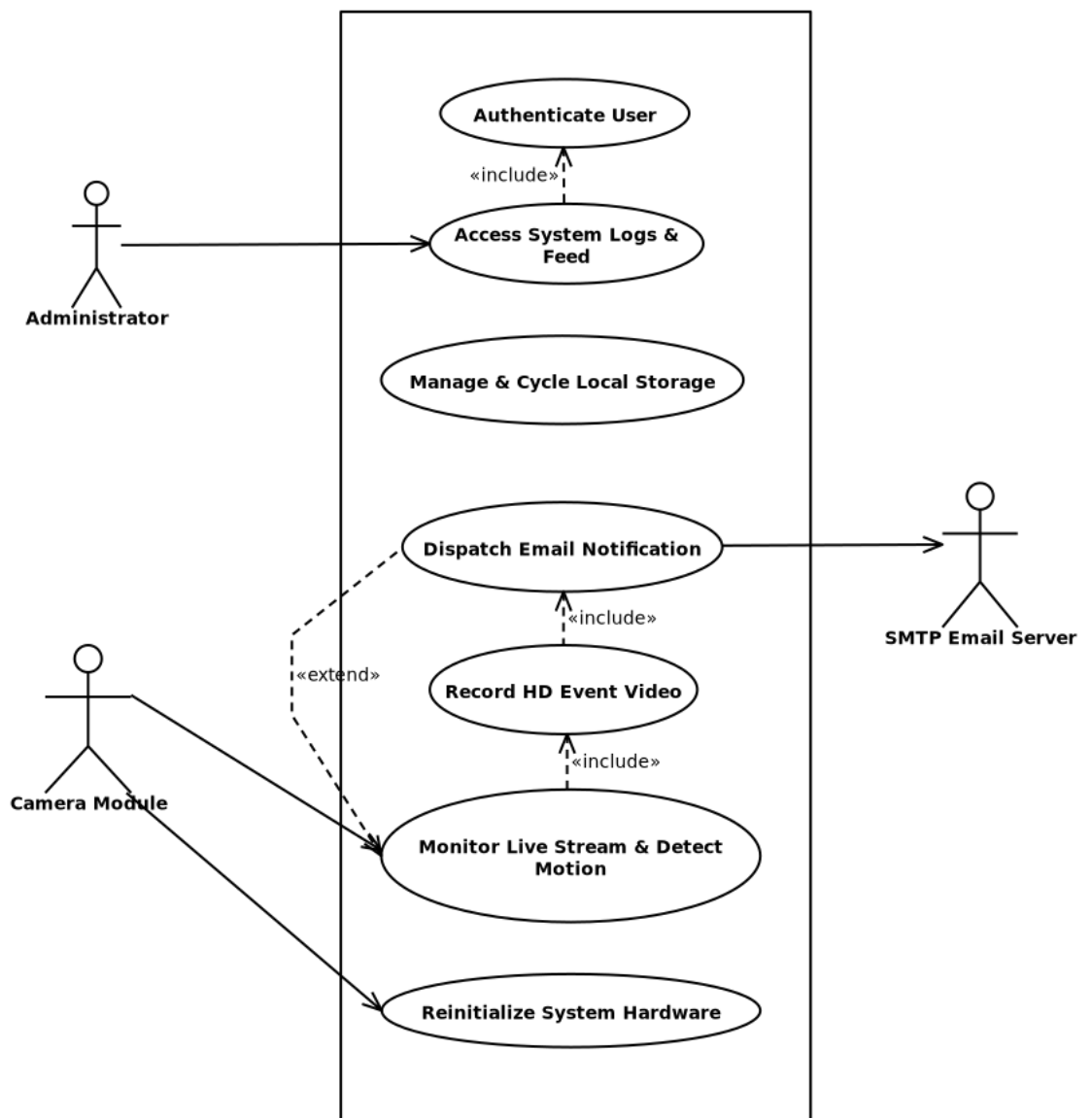
As a self-contained entity, the firmware manages the entire lifecycle of a security event locally. It directly controls the hardware abstraction layer of the camera module, processes the video stream frame-by-frame, and manages its own localized storage and alerting protocols. This independent design ensures that the system remains fully operational even during network isolation, establishing a secure, privacy-focused foundation for localized Internet-of-Things (IoT) security deployments.

### 2.2. Product Features

- **Real-Time Motion Detection:** Continuously processes the raw video stream frame-by-frame at the edge to detect localized visual anomalies.
- **Automated Video Recording:** Initiates high-definition local recording (1080p at 20fps) for a fixed window whenever motion thresholds are crossed.

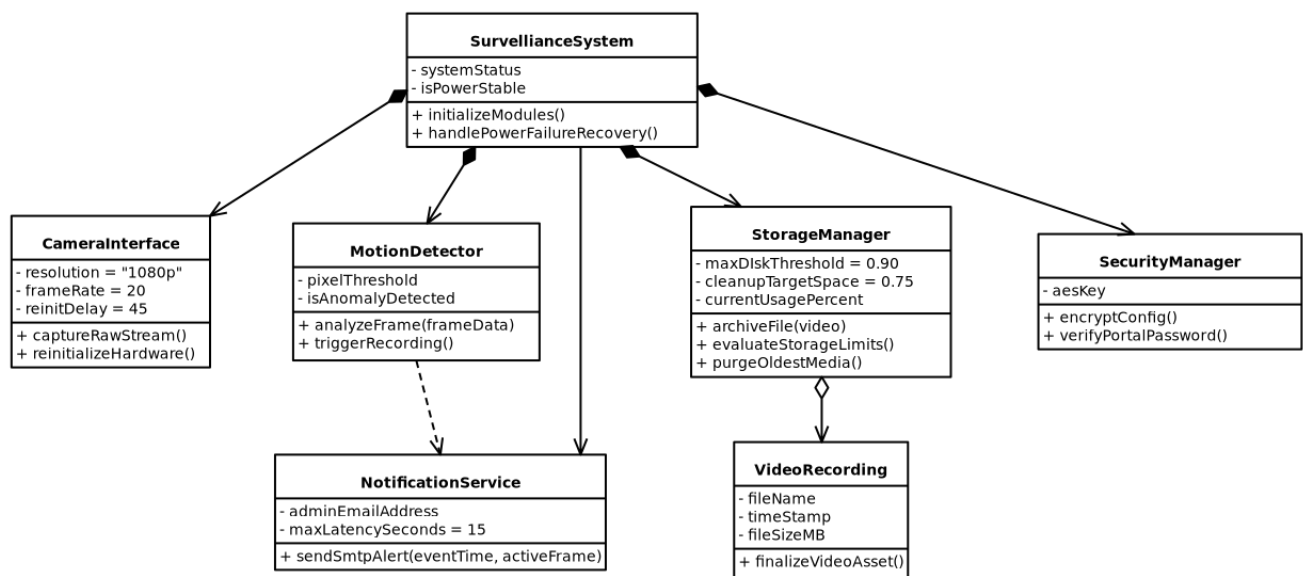
- **Decentralized Media Management:** Automatically formats, labels, and archives video files locally, with built-in rolling storage cleanup when disk usage peaks.
- **Instant Alert & Telemetry Dispatch:** Generates and distributes secure email alerts complete with precise timestamps and evidence images directly to an administrator.
- **Edge Data Security & Authentication:** Restricts system configuration access via hashed credentials and encrypts sensitive network keys locally.
- **Fault-Tolerant Power Recovery:** Self-heals and automatically reinitializes critical hardware components following electrical power disruptions.

uc Motion Detection Surveillance System



The internal firmware architecture of the Raspberry Pi Motion Detection Surveillance System is strictly object-oriented to ensure modularity and ease of maintenance. The system utilizes dedicated software interfaces to standardize edge device orchestration, while strictly separating the volatile, real-time frame processing and computer vision logic from the non-volatile local disk storage and rolling media management mechanics. The UML Class Diagram below illustrates the composition of the main surveillance controller, its tight encapsulation of peripheral hardware modules, and its operational media asset aggregation methods.

cls Surveillance System



## 2.3. User Classes and Characteristics

«Identify the various user classes (intended target for the product) that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.»

## 2.4. Operating Environment

«Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.»

## 2.5. Design and Implementation Constraints

«Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory

requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).»

## **2.6. User Documentation**

«List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.»

## **2.7. Assumptions and Dependencies**

«List any assumed factors (as opposed to known facts) that could affect the requirements stated in the specification. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).»

## **3. System Features**

This section provides a brief overview of the primary functional capabilities of the Motion Detection Surveillance System firmware. Each feature is described concisely here; detailed functional requirements, stimulus/response sequences, and implementation constraints are elaborated in Section 5.

### **3.1. Real-Time Motion Detection Engine**

The system shall continuously analyze the live video stream on a per-frame basis and classify a motion event whenever the pixel-level difference between two consecutive frames meets or exceeds the configured detection threshold. This capability forms the core trigger mechanism for all downstream recording and alerting operations.

### **3.2. Automated Video Capture and Local Archiving**

Upon a confirmed motion trigger, the system shall initiate a fixed-duration high-definition video recording and persist the resulting file to local non-volatile storage under a timestamp-based filename. The system shall operate this pipeline entirely at the edge, without dependency on any external network storage or cloud infrastructure.

### **3.3. Alert and Telemetry Dispatch**



The system shall generate and transmit an email notification to the pre-configured administrator address upon completion of each recording. The notification shall include the event timestamp and an embedded JPEG image corresponding to the most visually active frame captured during the motion event.

### **3.4. Rolling Media Lifecycle Management**

The system shall monitor local disk utilization on a continuous basis. When storage consumption exceeds a defined upper threshold, the system shall autonomously purge the oldest archived media files in chronological order until utilization falls below the defined lower threshold, ensuring uninterrupted recording capability without administrator intervention.

### **3.5. Fault-Tolerant Power Recovery**

The system shall detect loss and subsequent restoration of electrical power and shall execute a structured hardware reinitialization sequence following a fixed stabilization delay, ensuring all peripheral components have reached a stable operating state before surveillance operations resume.

### **3.6. Secure Configuration and Access Control**

The system shall enforce authentication for all administrative access and shall store sensitive configuration parameters — including SMTP credentials, Wi-Fi keys, and administrator contact details — in encrypted form in non-volatile memory. No plaintext credentials shall be written to disk at any point during system operation.

## **4. External Interface Requirements**

### **4.1. User Interfaces Overview**

The Motion Detection Surveillance System exposes a minimal, administrator-only web portal as its primary user-facing interface. Given the embedded, headless nature of the Raspberry Pi deployment, there is no physical display or local GUI. All human interaction occurs remotely via a browser-based interface served over the local network by the firmware's built-in web server.

The web portal provides the following high-level functions:

- **Authentication screen:** A login form requiring a username and SHA-256-validated password before granting access. All other portal views are inaccessible without a valid authenticated session.
- **Live stream view:** Displays the real-time MJPEG or HLS video feed from the camera module, allowing remote visual monitoring of the surveilled area.
- **Event log view:** Presents a paginated, timestamped list of past motion events, each linked to its associated recorded video file and alert image.
- **System configuration panel:** Allows the administrator to modify SMTP credentials, recipient email address, motion detection sensitivity, and recording duration

parameters. Input fields for sensitive values (passwords, Wi-Fi keys) shall be masked. All configuration changes must be confirmed before being persisted to encrypted storage.

The interface shall function correctly on any standards-compliant modern browser (Chrome, Firefox, Edge, Safari) without requiring additional plugins. Error states (e.g., camera unavailable, disk full warning) shall be surfaced via clearly visible status indicators on the portal dashboard.

## 4.2. Hardware Interfaces

- **Camera Module (Raspberry Pi Camera Module v2 / v3 or compatible)**

The firmware interfaces with the camera module through the Raspberry Pi's MIPI CSI-2 serial interface. Logically, the software communicates with the device via the `picamera2` Python library, which abstracts the low-level V4L2 kernel driver. The firmware configures the camera to operate in continuous preview mode, capturing raw frames at a resolution sufficient for 1080p output at 20fps. The firmware also monitors the hardware readiness state of the camera module during the post-power-failure recovery sequence (45-second stabilization window) before reinitializing the capture pipeline.

- **Local Storage (MicroSD Card / USB Drive)**

All video recordings and configuration data are persisted to the local block storage device mounted on the Raspberry Pi. The firmware interacts with the storage through the host operating system's standard POSIX file I/O interface. The system monitors available disk space and triggers the rolling cleanup routine (Section 3.4) autonomously.

- **Network Interface (Wi-Fi / Ethernet)**

The firmware uses the Raspberry Pi's onboard network interface (IEEE 802.11 Wi-Fi or 100/1000BASE-T Ethernet) to transmit alert emails via SMTP and to serve the administrator web portal over HTTP/HTTPS. Wi-Fi credentials are stored in encrypted form and loaded by the firmware at initialization.

## 4.3. Software Interfaces

### **Operating System — Raspberry Pi OS (Linux, Debian-based)**

The firmware runs as a user-space Python application on top of Raspberry Pi OS Lite. It relies on the OS for process scheduling, file system access, network stack, and hardware abstraction. No RTOS is used; real-time constraints are met through optimized frame-processing loops and threading.

### **Camera Driver — `picamera2` (libcamera backend)**

The firmware imports `picamera2` as its primary hardware abstraction interface to the camera module. This library wraps the `libcamera` framework and exposes a Python API for frame capture, resolution control, and hardware lifecycle management.

### **Computer Vision — OpenCV (`cv2`)**

Incoming frames are processed using OpenCV for pixel-level frame differencing and motion

threshold evaluation. OpenCV also handles JPEG encoding of alert images. The firmware uses OpenCV's frame subtraction and contour analysis routines to classify events.

#### **Email Dispatch — `smtpLib` / `email` (Python Standard Library)**

Alert emails are composed and transmitted via Python's built-in `smtpLib` interface using SMTP with TLS (port 587 or 465). The email payload is assembled using the `email.mime` module, with the JPEG alert image attached as a MIME multipart body.

#### **Web Server — Flask (or lightweight equivalent)**

The administrator web portal is served by an embedded Flask application running on the Raspberry Pi. Flask handles routing for the login, live stream, event log, and configuration endpoints. Session management and authentication state are handled server-side.

#### **Cryptographic Storage — `cryptography` library (AES-256) / `hashLib` (SHA-256)**

Sensitive configuration parameters are encrypted at rest using the `cryptography` Python library with AES-256-GCM. Web portal password validation is performed using SHA-256 digests via Python's built-in `hashLib` module. No plaintext credentials are stored on disk at any time.

## **5. System Features/Modules**

«This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product.»

### **5.1. «System Feature Name»**

#### **5.1.1 Description and Priority**

«Provide a short description of the feature. Give its priority, too.»

#### **5.1.2 Stimulus/Response Sequences**

«"Stimulus" or "Condition"»: «Describe the stimulus or condition»

Response: «Describe the response»

«Additional Stimulus/Response pairs as needed»

#### **5.1.3 Functional Requirements -----**

REQ-1.1: Whenever the System detects a 5% difference in pixels between two consecutive frames, the System should start recording a 1080p video with 20fps for a period of 15 seconds.

REQ-1.2: Once the 15 second recording time is completed, the System should save the recorded video in the local folder using the format YYYYMMDD\_HHMMSS.mp4.

REQ-1.3: After successfully saving a new video, the System should automatically send an e-mail notification containing a timestamp of the captured motion event along with a JPEG image of the most active frame to the configured administrator address within 10 seconds.

REQ-1.4: Once the local storage usage exceeds 90%, the System should remove oldest media files sequentially until the disk space usage drops below 75%. REQ-1.5: Once a system power

failure occurs, the System will reinitialize the camera module interface once it becomes ready after 45 seconds.

## **5.«N». «Additional feature blocks»**

## **6. Nonfunctional Requirements ----**

### **6.1. Performance**

**NF-1.1:** The motion detection engine should process incoming raw video stream coming from the Raspberry Pi

**NF-1.2:** The end to end latency from the moment physical motion was detected until its final delivery to the configured administrator address in an email should not exceed 15 seconds.

### **6.2. Security**

**NF-2.1:** Configuration payloads such as SMTP passwords, Wi-Fi keys, and administrator email parameters shall be securely stored in non-volatile memory using AES-256 encryption.

**NF-2.2:** User authentication using SHA-256 hashing algorithm for password validation shall be mandatory for the web portal through which system logs and live streaming feeds are accessed.

### **6.3. Reliability**

**NF-3.1:** 99.5% uptime shall be achieved by the system during any test period of 30 days except the time taken for site-wide electrical infrastructure maintenance.